

Titanic Survival Prediction using Machine Learning

Project objective :

The objective of this project is to build a Machine Learning model that predicts whether a passenger survived the Titanic disaster based on passenger information such as age, gender, ticket class, fare, and family details.

Import libraries :

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)
```

Load dataset:

```
dftrain = pd.read_csv("/content/sample_data/train.csv")
dftest = pd.read_csv("/content/sample_data/test.csv")
```

```
dftrain.shape
```

```
(891, 12)
```

```
dftest.shape
```

```
(418, 11)
```

```
dftrain.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     891 non-null    int64
1   Survived        891 non-null    int64
2   Pclass          891 non-null    int64
```

```

3   Name      891 non-null  object
4   Sex       891 non-null  object
5   Age       714 non-null  float64
6   SibSp     891 non-null  int64
7   Parch     891 non-null  int64
8   Ticket    891 non-null  object
9   Fare      891 non-null  float64
10  Cabin     204 non-null  object
11  Embarked  889 non-null  object
dtypes: float64(2), int64(5), object(5)
memory usage: 83.7+ KB

```

```
dftest.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   PassengerId     418 non-null   int64
1   Pclass          418 non-null   int64
2   Name            418 non-null   object
3   Sex             418 non-null   object
4   Age            332 non-null   float64
5   SibSp           418 non-null   int64
6   Parch           418 non-null   int64
7   Ticket          418 non-null   object
8   Fare            417 non-null   float64
9   Cabin           91 non-null    object
10  Embarked        418 non-null   object
dtypes: float64(2), int64(4), object(5)
memory usage: 36.1+ KB

```

```
dftrain.describe()
```

	PassengerId	Survived	Pclass	Age	SibSp	Parch
count	891.000000	891.000000	891.000000	714.000000	891.000000	891.000000
mean	446.000000	0.383838	2.308642	29.699118	0.523008	0.381594
std	257.353842	0.486592	0.836071	14.526497	1.102743	0.806057
min	1.000000	0.000000	1.000000	0.420000	0.000000	0.000000
25%	223.500000	0.000000	2.000000	20.125000	0.000000	0.000000
50%	446.000000	0.000000	3.000000	28.000000	0.000000	0.000000
75%	668.500000	1.000000	3.000000	38.000000	1.000000	0.000000
max	891.000000	1.000000	3.000000	80.000000	8.000000	6.000000

```
dftest.describe()
```

	PassengerId	Pclass	Age	SibSp	Parch	Fare
count	418.000000	418.000000	332.000000	418.000000	418.000000	417.000000
mean	1100.500000	2.265550	30.272590	0.447368	0.392344	35.627188
std	120.810458	0.841838	14.181209	0.896760	0.981429	55.907576
min	892.000000	1.000000	0.170000	0.000000	0.000000	0.000000
25%	996.250000	1.000000	21.000000	0.000000	0.000000	7.895800
50%	1100.500000	3.000000	27.000000	0.000000	0.000000	14.454200
75%	1204.750000	3.000000	39.000000	1.000000	0.000000	31.500000
max	1309.000000	3.000000	76.000000	8.000000	9.000000	512.329200

dftrain.isnull().sum()

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	177
SibSp	0
Parch	0
Ticket	0
Fare	0
Cabin	687
Embarked	2

dtype: int64

dftest.isnull().sum()

	0
PassengerId	0
Pclass	0
Name	0
Sex	0
Age	86
SibSp	0
Parch	0
Ticket	0
Fare	1
Cabin	327
Embarked	0

dtype: int64

```
dftrain['Age'] = dftrain['Age'].fillna(dftrain['Age'].median())
```

```
dftest['Age'] = dftest['Age'].fillna(dftest['Age'].median())
```

```
dftrain['Fare'] = dftrain['Fare'].fillna(dftrain['Fare'].median())
```

```
dftest['Fare'] = dftest['Fare'].fillna(dftest['Fare'].median())
```

```
dftrain['Embarked'] = dftrain['Embarked'].fillna(dftrain['Embarked'].mode()[0])
```

```
dftest['Embarked'] = dftest['Embarked'].fillna(dftest['Embarked'].mode()[0])
```

```
dftrain.drop(['Name','Cabin','Ticket'],axis = 1,inplace = True)
```

```
dftest.drop(['Name','Cabin','Ticket'],axis = 1,inplace = True)
```

```
dftrain.columns
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Sex', 'Age', 'SibSp',
      'Parch',
      'Fare', 'Embarked'],
      dtype='object')
```

```
dftest.columns
```

```
Index(['PassengerId', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch',  
      'Fare',  
      'Embarked'],  
      dtype='object')
```

```
dftrain.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 891 entries, 0 to 890  
Data columns (total 9 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   PassengerId     891 non-null    int64  
1   Survived        891 non-null    int64  
2   Pclass          891 non-null    int64  
3   Sex             891 non-null    object  
4   Age            891 non-null    float64  
5   SibSp           891 non-null    int64  
6   Parch           891 non-null    int64  
7   Fare            891 non-null    float64  
8   Embarked        891 non-null    object  
dtypes: float64(2), int64(5), object(2)  
memory usage: 62.8+ KB
```

```
dftest.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 418 entries, 0 to 417  
Data columns (total 8 columns):  
#   Column          Non-Null Count  Dtype  
---  ---  
0   PassengerId     418 non-null    int64  
1   Pclass          418 non-null    int64  
2   Sex             418 non-null    object  
3   Age            418 non-null    float64  
4   SibSp           418 non-null    int64  
5   Parch           418 non-null    int64  
6   Fare            418 non-null    float64  
7   Embarked        418 non-null    object  
dtypes: float64(2), int64(4), object(2)  
memory usage: 26.3+ KB
```

Exploratory Data Analysis

```
le = LabelEncoder()  
dftrain["Sex"] = le.fit_transform(dftrain["Sex"])  
dftest["Sex"] = le.transform(dftrain["Sex"])
```

```
dftrain.head()
```

	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	0	3	1	22.0	1	0	7.2500	
1	2	1	1	0	38.0	1	0	71.2833	
2	3	1	3	0	26.0	0	0	7.9250	
3	4	1	1	0	35.0	1	0	53.1000	
4	5	0	3	1	35.0	0	0	8.0500	

```
le2 = LabelEncoder()
dftrain["Embarked"] = le2.fit_transform(dftrain["Embarked"])
dftest["Embarked"] = le2.transform(dftest["Embarked"])
```

```
dftrain.head()
```

	PassengerId	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	0	3	1	22.0	1	0	7.2500	
1	2	1	1	0	38.0	1	0	71.2833	
2	3	1	3	0	26.0	0	0	7.9250	
3	4	1	1	0	35.0	1	0	53.1000	
4	5	0	3	1	35.0	0	0	8.0500	

Building X and Y :

```
X = dftrain.drop("Survived",axis = 1)
Y = dftrain["Survived"]
```

```
X.head()
```

	PassengerId	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked
0	1	3	1	22.0	1	0	7.2500	2
1	2	1	0	38.0	1	0	71.2833	0
2	3	3	0	26.0	0	0	7.9250	2
3	4	1	0	35.0	1	0	53.1000	2
4	5	3	1	35.0	0	0	8.0500	2

```
Y.head()
```

Survived	
0	0
1	1
2	1
3	1
4	0

dtype: int64

Train-Test Split :

```
X_train,X_test,Y_train,Y_test = train_test_split(  
X,  
Y,  
test_size = 0.2,  
random_state = 42  
)
```

X_train.shape

(712, 8)

X_test.shape

(179, 8)

Y_train.shape

(712,)

Y_test.shape

(179,)

Model Making :

```
model = LogisticRegression(max_iter = 1000)
```

Model Train :

```
model.fit(X_train, Y_train)
```

```
▼ LogisticRegression ⓘ ?  
LogisticRegression(max_iter=1000)
```

Prediction :

```
Y_pred = model.predict(X_test)
```

Accuracy :

```
accuracy = accuracy_score(Y_test,Y_pred)  
print("accuracy:", accuracy)
```

```
accuracy: 0.8100558659217877
```

Confusion Matrix :

```
print(confusion_matrix(Y_test,Y_pred))
```

```
[[90 15]  
 [19 55]]
```

Classification Report :

```
print(classification_report(Y_test,Y_pred))
```

	precision	recall	f1-score	support
0	0.83	0.86	0.84	105
1	0.79	0.74	0.76	74
accuracy			0.81	179
macro avg	0.81	0.80	0.80	179
weighted avg	0.81	0.81	0.81	179

Confusion Matrix Visualization :

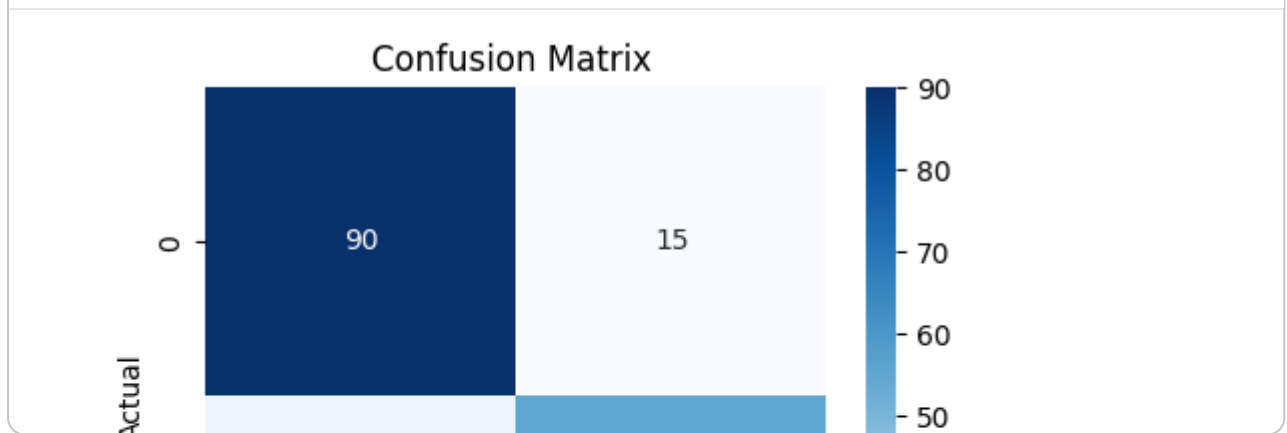
```
cm = confusion_matrix(Y_test,Y_pred)  
print(cm)
```

```
[[90 15]  
 [19 55]]
```

```
plt.figure(figsize=(5,4))  
sns.heatmap(cm,annot = True,fmt = 'd',cmap = 'Blues')  
plt.xlabel("predicted")  
plt.ylabel("Actual")
```



```
plt.title("Confusion Matrix ")  
plt.show()
```



Conclusion:

A Logistic Regression model was developed to predict passenger survival on the Titanic dataset. After preprocessing the data and training the model, it achieved an accuracy of 81.01% on the test dataset. This result indicates that the model performs well in predicting passenger survival based on features such as passenger class, sex, age, fare, and family information. Further improvements can be achieved by using advanced machine learning algorithms and feature engineering techniques.